

2주차: LoRA, RAG

LoRA: Low-Rank Adaptation of Large Language Models

1. 문제 (Problem Statement)

기존 LLM을 downstream task에 맞게 파인튜닝하려면 **모든 파라미터를 업데이트**해야 함.

- 모델 변화: $\Phi_0 \rightarrow \Phi_0 + \Delta\Phi$ (업데이트 크기 = 원본 모델 크기)
- GPT-3 기준으로 task마다 **175B 모델을 따로 저장**해야 하는 문제 발생

2. 기존 해결책의 한계

방법	문제점
Adapter Layer 추가	순차 처리 강제 → GPU 병렬화 불가, inference latency 증가
Prefix / Prompt Tuning	성능이 파라미터 수에 비선형적으로 의존, 사용 가능한 sequence 길이 감소

3. LoRA의 핵심 아이디어

모델 적응 과정에서 **weight 변화는 low-rank 구조를 가진다.**

기존 파인튜닝은 $W = W_0 + \Delta W$ 인데, LoRA는 ΔW 를 두 개의 작은 행렬로 분해함.

$$h = W_0 x + \Delta W x = W_0 x + B A x$$

$W_0 \in \mathbb{R}^{(d \times k)}$ → 기존 weight (freeze)
 $B \in \mathbb{R}^{(d \times r)}$ → 학습 대상
 $A \in \mathbb{R}^{(r \times k)}$ → 학습 대상
 $r \ll d, k$ → rank (매우 작은 값)

- W_0 는 **freeze**, A와 B만 학습
- A는 가우시안 초기화, B는 0으로 초기화 (학습 초기에 $\Delta W = 0$ 보장)

4. LoRA 적용 위치 & 실용적 이점

Transformer Self-Attention의 W_q , W_v 에만 적용 (MLP layer는 freeze)

Feature	Before	After
Memory Usage	1.2TB	350GB
Checkpoint Size	350GB	35MB
Training Speed	-	~25% faster
Inference Latency	-	추가 없음 (W_o 에 병합)

추론 시 $W = W_o + BA$ 로 병합하면 구조 변화 없이 기존 모델과 동일하게 동작

5. 실험 결과 (Empirical Experiments)

NLU (자연어 이해) — GLUE Benchmark

- RoBERTa, DeBERTa 기반 실험
- LoRA가 Full Fine-tuning과 동등하거나 그 이상의 성능 달성

NLG (자연어 생성) — GPT-2, GPT-3

- GPT-3 (175B) 기준 단 4.7M 파라미터만 학습
- WikiSQL, MultiNLI에서 최고 성능 달성

6. Low-Rank 구조의 타당성 검증

rank 크기에 따른 성능 변화

- $r=1$, $r=2$ 일 때도 $r=64$ 와 성능 차이가 거의 없음
- 단, 모든 task에서 작은 r 이 최적은 아님 (task 의존적)

Subspace 유사도 분석

- rank 8 모델과 rank 64 모델의 학습 방향이 거의 동일 ($\phi \geq 0.5$)
- 다른 random seed로 학습해도 비슷한 방향으로 수렴
- → 모델이 업데이트할 중요한 방향은 이미 정해져 있음을 시사

기존 weight와의 관계

- LoRA 업데이트(ΔW)는 기존 weight(W)와 **연관성이 높음**
- 기존 weight와 다른 방향의 특징을 **크게 증폭**시키는 방식으로 작동

7. 결론 & 향후 연구

Conclusion

- Low-rank 기반으로 파라미터 효율적 적응 달성
- 추가 inference latency 없이 적용 가능
- Shared parameter 구조로 빠른 task switching 가능

Future Work

- 다른 PEFT 방법과의 결합
- LoRA 적용 layer 선택 자동화
- Model weight의 low-rank 구조에 대한 추가 분석

RAG: Retrieval-Augmented Generation

0. 배경 (Background)

기존 LLM의 3가지 핵심 한계

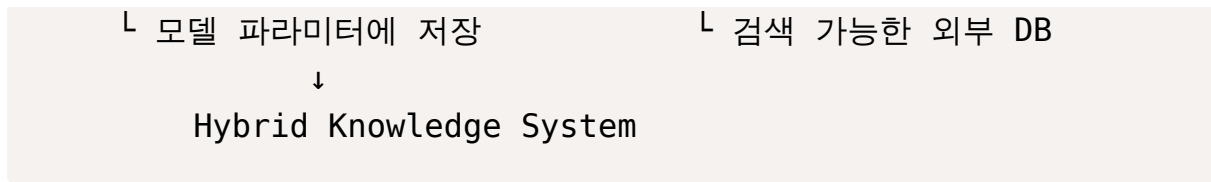
한계	설명
메모리 수정 불가	학습 후 지식을 쉽게 추가/수정 못함
파라미터 한계	모든 지식을 파라미터에 저장하기 어려움
할루시네이션	없는 사실을 만들어냄

1. RAG란?

외부 문서 DB에서 관련 정보를 검색(Retrieve)한 뒤, LLM과 결합하여 답변을 생성(Generate)하는 방식

Hybrid Memory 구조

Parametric Memory (BART) + Non-Parametric Memory (Wikipedia)



2. 구성 요소

Retriever — DPR (Dense Passage Retrieval)

- **Bi-encoder 구조**: 쿼리와 문서를 각각 독립적으로 인코딩 후 내적으로 유사도 계산
- 전체 Wikipedia 문서를 임베딩하여 **비모수적 메모리(문서 인덱스)** 구축
- TriviaQA, Natural Questions 기반으로 학습

Generator — BART

- 4억 파라미터의 사전 학습된 seq2seq 트랜스포머
- 쿼리 x + 검색된 문서 z 를 단순 연결하여 입력
- 인코더-디코더 구조로 자연스러운 텍스트 생성

Training

- 쿼리 인코더(**BERT_q**) + **BART 생성기**를 end-to-end fine-tuning
- 문서 인코더(**BERT_d**)와 인덱스는 **고정** (재구축 비용 대비 성능 향상 미미)

3. Decoding 방식 2가지

	RAG-Sequence	RAG-Token
핵심	문서 하나를 전체 시퀀스에 사용	토큰마다 다른 문서 참조 가능
유연성	낮음	높음
강점	단순하고 일관된 답변	복잡한 지식 통합

4. 실험 결과

Open-domain QA (NQ, TriviaQA, WebQ, CuratedTrec)

- RAG-Seq가 NQ, CT에서 **SOTA 달성**
- TriviaQA에서 RAG-Token이 **56.8 / 68.0**으로 최고 성능

- 단순한 구조에서도 복잡한 extractive 모델과 closed-book 모델을 능가

Abstractive QA (MSMARCO)

- BART 단독 대비 **+2.6점 향상**
- Gold passage 없이도 자연스러운 답변 생성
- BART는 할루시네이션이 발생하지만 RAG는 사실 기반 생성

Jeopardy Question Generation

- RAG-Token이 인간 평가에서 **Factuality 42.7%, Specificity 37.4%** 우세
- 여러 문서를 결합하는 RAG-Token > 단일 문서 기반 RAG-Sequence

Fact Verification (FEVER)

- SotA 대비 성능은 약간 낮지만, **supervision 없이 스스로 evidence를 탐색**
- Top-1 evidence retrieval: **71%**, Top-10: **90%**

5. 검색 문서 수(k)의 영향

모델	k 증가 시 QA 성능
RAG-Sequence	꾸준히 향상
RAG-Token	k=10에서 최대, 이후 감소

- k가 클수록 **Recall 향상** (정답 문서 포함 확률 증가)
- 생성 품질(BLEU-1, Rouge-L)도 k 증가 초기에 빠르게 향상

6. 결론 & 향후 연구

Conclusion

┃ RAG = Parametric Memory + Non-Parametric Memory의 결합

Future Work

- Parametric ↔ Non-parametric memory의 상호작용 이해
- Retriever와 Generator를 처음부터 함께 pre-training하는 방법 탐구
- 다단계 RAG / 대화형 RAG / 에이전트 RAG 등으로 발전